

ONE RUN, FUNCTION BY FUNCTION — the complete call tree

This document follows a single run of the factory in time order, top to bottom. Every box below is a real function; indentation means "called by the function above". Read it like a map: find a name here, then open that function in the code.

0. HOW TO READ THIS

Indentation = "called by". Order on the page = order in time. **LLM** = an AI agent is called here (the only places AI is involved). **SUBPROCESS** = a separate Python process is spawned. **THREADS** = work fans out to parallel threads. **writes** / **reads** = files touched. Everything else is plain Python in the main thread — which is most of it, by design.

1. BOOT — FROM YOUR COMMAND TO A READY WORKSPACE

```
main() run_factory.py:1073 — the entry point. Parses flags, initialises if needed, runs the orchestrator, converts exceptions to exit codes.
├─ (before main, at import: if not already inside .venv, re-exec the same script with .venv's python — torch lives there; lines 25–32)
├─ is_initialized() init_workspace.py:29 — True if state_space.json AND factory_state.json both exist
├─ initialize(fresh) init_workspace.py:33 — runs only with --fresh or on the very first run (details below)
├─ FactoryOrchestrator(max_iterations, ...) run_factory.py:105 — loads the world:
│   ├── read_json(FACTORY_STATE_PATH) state_store.py:10 reads factory_state.json — the scorebook
│   ├── StateSpace(read_json(STATE_SPACE_PATH)) state_space.py:100 reads state_space.json — the search space + ledger
│   ├── ModelRouter() router.py:108 — the L0–L3 model ladder + rules R1–R6
│   ├── IsolationGuard(state["canaries"]) isolation.py:33 — per-experiment leak detection
│   ├── ClaudeCLIClient(log) llm_client.py — the one door to the LLM (Claude CLI)
│   └─ AgentRunner(llm, router, guard, usage_cb, log) agents.py:84 — every agent call in the run passes through this
└─ orch.run() run_factory.py:191 — the main loop (page 2). Wrapped in try/except:
    ├── MacroFail → log + writes workspace/macro_fail.json → exit code 2 (abort, do not resume)
    ├── InfraUnavailable → save_state() → exit code 3 (pause; rerun the same command to resume)
    └─ KeyboardInterrupt → save_state() → exit code 130 (pause, resumable)
```

```
initialize() init_workspace.py:33 — first run only (or --fresh). Builds the workspace from the seed.
├─ --fresh: move any existing workspace to experiments/_archived/<timestamp>
├─ copy the seed writes factory/templates/state_space_seed.json → workspace/state_space.json (one dimension: hidden_layers, one value [64,64])
├─ copy the paper skeleton writes → workspace/research_paper.md
├─ writes workspace/best_config.json + workspace/last_judge.json (empty placeholders)
├─ protect_ai_docs() init_workspace.py:23 — mark ai_docs/ read-only
└─ write_json(FACTORY_STATE_PATH, new_factory_state(ai_docs_checksum()))
    └─ new_factory_state() state_store.py:30 writes factory_state.json: next_iteration=0, empty iterations/scores/poisoned, best=None, llm_usage={calls:0, cost:0},
    ai_docs_checksum
```

2. THE CAST BEHIND THE CURTAIN (CALLED FROM EVERYWHERE, SHOWN ONCE)

Helper	Lives in	What it does
read_json / write_json / write_text	state_store.py	Every file read/write in the system goes through these three. JSON with a default on read; atomic-ish writes.
AgentRunner.run(call, rlog)	agents.py:91	The one function that talks to an LLM (full anatomy on page 4). Render prompt → route → call → validate → retry.
ModelRouter.route(ctx)	router.py:113	Decides which model + effort each call gets (rules R1–R6, ladder L0–L3).
EscalationTracker.escalate(agent)	router.py:96	Per-iteration counter: each failure bumps that agent one rung up the ladder (cap L3).
IsolationGuard.check(...)	isolation.py:37	Scans the outgoing prompt for canary tokens / foreign experiment ids. A leak is a MacroFail — run aborted.
StateSpace.apply_wave_proposal(...)	state_space.py:113	The Meta validator: checks mutations are legal, coordinates in-domain and unvisited; returns the new space + coordinate hashes.

The main loop and **PHASE A** — Meta proposes the wave

3. RUN() — THE LOOP ITSELF IS TINY

FactoryOrchestrator.run() `run_factory.py:191` — check stop conditions, run one wave, check the circuit breaker. Repeat.

```
run() run_factory.py:191
├ already halted? log and exit 0 (a finished factory stays finished; use --fresh to restart)
├ preflight() run_factory.py:148 — runs once per process start:
│   ├── required dirs exist (ai_docs, prompts, templates) — else MacroFail
│   ├── ai_docs checksum unchanged since init — else ReadOnlyViolation → MacroFail
│   ├── sim_harness.ensure_cache() simulator/harness.py:27 — build the scenario cache if missing
│   ├── sim_harness.warm_up() simulator/harness.py:116 — import torch + load cache once, so no simulation pays the cold start
│   ├── load_cache()["baselines"] → store mean baseline returns in state["baselines"]
│   └ crash recovery: experiments numbered ≥ next_iteration are leftovers of an interrupted wave → move to _aborted/, restore .snapshots/*before_NNN.json, rebuild guard, writes
state
└ loop forever:
    ├── done_checker.check(state, max_iterations) done_checker.py:7 — Python decides, before any agent runs. Returns a reason string or None:
    │   ├── "max-iterations reached (N/50)"
    │   ├── "state space stagnant: no dimension added in the last 10 iterations..."
    │   └ "score plateau: best of last 10 iterations = X, best before = Y; improvement ≤ 2.0"
    ├── reason → halt(reason) run_factory.py:1002 (page 5) → exit 0
    ├── no reason → run_wave() run_factory.py:232 (the rest of this document)
    └ circuit_breaker() run_factory.py:206 — if the last 2 waves were entirely poisoned by ONE error type: that's a systematic fault, not bad luck → raise InfraUnavailable
(pause, resumable)
```

4. RUN_WAVE() — SETUP

run_wave() `run_factory.py:232` — one wave = K iterations sharing one Meta proposal. Phases A–E.

```
├ start = state["next_iteration"]; width = min(parallel, remaining budget); indices = [start ... start+width-1]
├ writes .snapshots/state_space.before_NNN.json + factory_state.before_NNN.json — the undo point for crash recovery
├ WaveContext(wave, last_judge_score, meta_invalid_streak) — the shared inputs to routing
└ _new_member(start) run_factory.py:225 — mkdir exp_NNN/prompts/, mint a canary (leak token), create the Member record (status "open") writes state["canaries"]
```

A phase_meta(lead, indices, wctx) `run_factory.py:304` — ONE Meta call proposes mutations + K coordinates for the whole wave. Serial. Up to 3 attempts.

```
├ payload: ITERATION, WAVE_SIZE, STATE_SPACE, BEST_CONFIG, LAST_JUDGE (all sanitized), REFILL_NOTE=""
├ for attempt in 0..2:
│   ├── runner.run(AgentCall(agent="meta", validator=..., on_failure=...), lead.rlog) agents.py:91
│   │   └ validator = space.apply_wave_proposal(out, indices) state_space.py:113 — legal mutations? coordinates in-domain, unvisited? → new space + hashes
│   ├── MicroFail → record in attempts.json, esc.escalate("meta") (+1 rung), on_failure bumps meta_invalid_streak, try again
│   ├── OSError → InfraUnavailable (pause, never poison). Other surprise → stage_error_meta_aN.txt + HarnessStageError (a MicroFail)
│   └ success → meta_invalid_streak = 0, adopt new space writes state_space.json; if a dimension was added: last_dimension_added_iteration = start
├ all 3 attempts failed → lead.status = "poisoned", return None → wave closes early with only the lead (poison bookkeeping + poison section, then phase E)
└ _fan_out_meta(lead, members, proposal) run_factory.py:368
    ├── writes exp_NNN/next_coordinate.json for every member: coordinate, hash, hypothesis, wave position, mutations applied
    ├── copies the lead's Meta routing entries into each member's router_log (the wave's one Meta call belongs to everyone's history)
    └ writes exp_lead/meta.json (the raw proposal)
```

PHASE B — build, fingerprint, simulate, judge (parallel)

B1 `_parallel(member_core, BUILD_STAGES)` `run_factory.py:286` **THREADS** — one thread per member, stages ("worker", "fingerprint"). Threads only compute; no shared state is written here.

- └ `_parallel(fn, members)` — `ThreadPoolExecutor`, join in iteration order, re-raise the first macro/infra error after ALL threads finish
- └ `member_core(m, stages)` `run_factory.py:393` — the 3-attempt retry machine for one iteration:
 - └ while `consecutive_fails < 3`: run each stage not already in `m.memo` (finished stages are memoised — a retry never redoes them)
 - └ `MicroFail` → `_record_failure()` `run_factory.py:521` **writes** `attempts.json`, escalate the failing agent +1 rung; if `SimulatorError/WorkerCodeError`: forget worker+fingerprint memo, keep the error text as `RETRY_ERROR` for the next worker prompt
 - └ all stages done (with judge) → `m.status = "ready"`
 - └ 3 fails → `m.status = "poisoned"` (the wave continues; this iteration is skipped)

stage_worker(m, attempt, last_score) `run_factory.py:548` — the Worker writes model.py.

- └ `_ai_docs()` `run_factory.py:533` **reads** `ai_docs/*.md` — the domain knowledge every worker sees
- └ payload: `COORDINATE`, `DIMENSIONS` (definitions of the dims in this coordinate), `AI_DOCS`, `RETRY_ERROR` (only after a failed attempt)
- └ `runner.run(AgentCall(agent="worker", validator=...))` **LLM** — validator = `check_model_source(model_py)` `code_guard.py` — the static gate: required functions present, no banned constructs
- └ **writes** `model.py` (with attribution header), `config.json` (coordinate, hypothesis, sha256 of `model.py`, static-gate report), `worker.json`

stage_fingerprint(m) `run_factory.py:429` — what did the worker actually build?

- └ `sim_harness.run_fingerprint(m.d, m.canary)` **SUBPROCESS** `simulator/harness.py:72` — loads `model.py` in a clean subprocess, extracts features/architecture/init/training/loss as a hash; also applies the validity gates so a silently broken model fails HERE, before simulation time is spent **writes** `fingerprint.json`

B2 `_dedupe(members)` `run_factory.py:435` — serial, main thread, iteration order. Two models with the same fingerprint are the same model — don't pay for it twice.

- └ for each member: fingerprint already seen (this run's state["fingerprints"] or an earlier wave member)?
- └ yes, and `refills < 2` → `_refill(m, dup, members)` `run_factory.py:457` **LLM** — ask Meta for ONE replacement coordinate (mutations forbidden); rejected coordinate saved as `next_coordinate.rejected_k.json`; rebuild memo; re-run B1 stages for this member
- └ refill failed or cap reached → `m.duplicate_of = dup` (still simulated — but flagged as a functional duplicate)

B3 `_parallel(member_core, CORE_STAGES)` **THREADS** — same retry machine, now with stages ("worker", "fingerprint", "simulate", "judge"); worker+fingerprint come from memo instantly.

- └ `stage_simulate(m)` `run_factory.py:604`
 - └ `sim_harness.run_simulator(m.d, canary, timeout)` **SUBPROCESS** `simulator/harness.py:31` — trains the model over 50 turns in a separate process with fixed env (`PYTHONHASHSEED=0`, no GPU). Then the harness validates the OUTPUT: exactly 50 ordered `TURN` lines, canary present, `SUMMARY` line, `sim_score` in `[0,100]`. Any violation → `SimulatorError` (a `MicroFail` — the worker gets another attempt). **writes** `simulation.log`, `sim_metrics.json`
- └ `stage_judge(m, attempt, last_score)` `run_factory.py:611`
 - └ payload: `CONFIG` (`config.json` minus the hypothesis — withheld so it can't bias the score), `SIMULATION_LOG` **reads**
 - └ `runner.run(AgentCall(agent="judge", validator=...))` **LLM** — validator: score must be an int, and `|score - sim_score| ≤ 3` (the anchor rule — the Judge grades the log, but may not drift from the machine)
 - └ **writes** `judge.json` = the verdict + a `harness` block (`sim_score`, `anchor_delta`, `tolerance`)

Why memoised stages matter: if the judge rejects attempt 2, the retry reuses the memoised worker, fingerprint and simulate results — only the judge stage re-runs. If the simulator says the code is broken, worker+fingerprint are forgotten and only those rebuild. Retries are surgical, never a full redo.

5. AGENTRUNNER.RUN() — EVERY LLM CALL, ANYWHERE, GOES THROUGH THIS

AgentRunner.run(call, router_log) `agents.py:91` — **prompt** → **route** → **guard** → **call** → **validate** → (in-step retry). This is the only place the factory talks to an AI.

- render(`user_template`, `payload`) `agents.py:54` **reads** `prompts/<agent>.system.md` + `<agent>.user.md` from disk
- `schema = schemas.BY_NAME[(agent, purpose)]` — the JSON contract for this call
- step-retry loop (STEP_RETRIES+1 tries, in place):
 - `router.route(ctx)` `router.py:113` — which model + effort? (R1 baseline, R3 quality, R4 complexity, R6 escalation, R2 step-retry effort bump, R5 fable-only-format)
 - `guard.check(...)` `isolation.py:37` — scan the outgoing prompt for canaries / foreign experiments. Leak → MacroFail, run over.
 - writes** `prompts/<agent>_<purpose>_aN_rM.md` — the exact routed prompt, on disk, auditable
 - `llm.call(decision, system, prompt, schema)` **LLM** `llm_client.py` — the actual API call (claude CLI)
 - writes** `prompts/...response.json` (raw response + cost) · `usage_cb` → `state["llm_usage"]`
 - `extract_json()` `agents.py:62` → `scrub_foreign()` `isolation.py:88` — parse the JSON; replace any foreign experiment mentions with "a prior experiment"
 - `jsonschema.validate(out, schema)` → then the stage's own validator (the ones you saw in phases A and B)
 - rejected → `router_log.record("failed")`, `on_failure()`, append the rejection to the prompt, retry with effort bumped (R2)
 - accepted → `router_log.record("ok")`, return the JSON. All step-retries exhausted → raise the error to the stage's retry machine (page 3)

Two retry levels, don't confuse them: the **step-retry** (inside AgentRunner) fixes one malformed answer in place, same attempt. The **attempt loop** (in `member_core/phase_meta`) is the 3-strikes machine that escalates agents up the ladder and eventually poisons. Step-retry is a rephrasing; attempts are new chances.

6. PHASE C — EVERYTHING LANDS, SERIALY, IN ITERATION ORDER

C stage_update(m) `run_factory.py:644` — **main thread only. This is where a thread's private results become the factory's official record.**

- for each member, in iteration order: `ready` → `stage_update(m)`; else → `_poison_bookkeeping(m)`
- writes** `state["scores"][i]`, `state["sim_scores"][i]`, `state["last_judge_score"]`
- `space.record(i, coord, "scored", score, sim_score)` `state_space.py:242` **writes** `state_space.json` ledger
- new best? (higher judge score, or equal score with higher `sim_score`) → `state["best"] = ...` → `_sync_best_files()` `run_factory.py:714` **writes** `workspace/best_model.py` + `best_config.json` (sanitised)
- save_state()** **writes** `factory_state.json`

_poison_bookkeeping(m) `run_factory.py:682` + **_write_last_judge(members)** `run_factory.py:662`

- poisoned: `space.record(i, coord, "poisoned", None)` (never proposed again) + append to `state["poisoned"]` with the failure reasons **writes**
- writes** `workspace/last_judge.json` = `{"previous_wave": [verdicts]}` — the wave's scores, feedback, next_steps and metric digests; this is what the NEXT wave's Meta reads

PHASE D + E — paper, close, and the halt

D _parallel(member_doc, scored) `run_factory.py:739` **THREADS** — one Doc agent per scored iteration; all see the SAME paper snapshot (taken before the phase starts).

- └ `stage_doc(m, attempt, paper_snapshot)` `run_factory.py:761`
 - └ draft call **LLM** — payload: ITERATION_LABEL, CONFIG, JUDGE, SIM_SUMMARY (summary_lines `simulator/harness.py:125` = log minus per-turn rows), PAPER. Validator: section starts "### Iteration NNN:", no H1/H2 headings.
 - └ `paperize()` `isolation.py:105` — scrub experiment ids from the section
 - └ format call (doc/format, always fable-5.1 — rule R5) **LLM** — markdown only; validator `paper.format_preserves()` `paper.py:136` rejects any content change, draft kept instead
 - └ `writes` doc.json (draft + format status)
 - └ 3 failures → NOT poisoned (the experiment is already scored) → `_fallback_section()` `run_factory.py:811` — Python writes a deterministic section from memo
- └ Poisoned members get `_poison_section()` `run_factory.py:826` — a deterministic "what failed and why" section. Every iteration appears in the paper, scored or not.

E _finish_wave(members, wave_no) `run_factory.py:874` — serial, main thread. The barrier where the wave becomes permanent.

- └ `paper.upsert_iteration(text, m.i, section)` `paper.py:41` × each member — insert/replace its section `writes` `research_paper.md`
- └ `paper.replace_block()` `paper.py:29` — ABSTRACT + RESULTS_NARRATIVE (from the latest draft), RESULTS_TABLE (`_leaderboard :863`), APPENDIX_SPACE (`paper.space_evolution` `paper.py:91`), STATUS (`_status_line :868`)
- └ register new fingerprints in state["fingerprints"]
- └ `_row(m)` `run_factory.py:843` + wave/router_verdict/closed_at → append to state["iterations"]; next_iteration = last i + 1; waves += 1 → `save_state()` `writes`
- └ `analytics.compute(state, space, parallel)` `analytics.py:62` → space.data["analytics"] (what works, per-dimension stats) → `save_space()` `writes`
- └ `verify_run.check_experiment(i, ...)` `verify_run.py:54` × each member → `writes` `exp_NNN/iteration_summary.json` (row + micro_done checks)
- └ delete the wave's .snapshots — the undo point is no longer needed; the wave is committed

7. HALT(REASON) — HOW A RUN ENDS (WHEN DONE_CHECKER SAYS SO)

halt(reason) `run_factory.py:1002` — final scoring, final writing, final report.

- └ `_confirm_top()` `run_factory.py:941` **SUBPROCESS** xK — re-score the statistically-tied top iterations on 200 hidden scenarios (never used for selection) → `confirmation.json` + `best_model_confirmed.py`
- └ `_factory_summary(reason)` `run_factory.py:915` — the run in numbers (iterations, waves, scores, best, llm_calls)
- └ final synthesis call (doc/final) **LLM** — writes ABSTRACT, RESULTS_NARRATIVE, DISCUSSION, CONCLUSION. Fails → deterministic closing text (`get_block_safe :1066`)
- └ final `paper.replace_block` pass + CONFIRMATION table + "Status: complete..." `writes` `research_paper.md`
- └ ai_docs checksum verified one last time (ReadOnlyViolation if changed mid-run)
- └ `writes` state["halted"] = {reason, at, iterations} → `save_state()`
- └ `verify_run.build_report()` `verify_run.py:149` `writes` `workspace/factory_report.json` (macro/micro verification) → exit 0

8. THE WHOLE RUN IN NINE LINES

```
main → initialize? → FactoryOrchestrator → run → preflight → loop { done_checker.check → run_wave → _circuit_breaker } → halt
run_wave = A phase_meta → _fan_out_meta → B1/B3 _parallel(member_core) → B2 _dedupe(→_refill) → C stage_update/_poison_bookkeeping → D
_parallel(member_doc) → E _finish_wave
member_core = stage_worker → stage_fingerprint → stage_simulate → stage_judge, inside a 3-attempt memoised retry machine
```

Read order suggestion: `run_factory.py` lines 1073 (main) → 191 (run) → 232 (run_wave) → 393 (member_core) → 548/604/611 (stages) → 644 (stage_update) → 874 (_finish_wave) → 1002 (halt). That path — about 300 lines — is the entire factory. Everything else in the file exists to serve those nine functions.